

Loops

28 July 2026 18:20

We often need to repeat actions.

Loops are a way to repeat same code multiple times.

The "while" loop

Syntax:

```
while(<condition>){  
  //code  
  //so-called "loop body"  
}
```

```
const prompt = require('prompt-sync')();  
let numOfTerms = Number(prompt('Enter number of terms: '));  
let i = 0;  
while(i < numOfTerms){  
  console.log(i)  
  i++;  
}  
console.log(`\nValues of j`)  
let j = 3;  
while(j){  
  console.log(j);  
  j--;  
}
```

Output:

Enter number of terms: 5

0
1
2
3
4

Values of j

3
2
1

A single execution of the loop body is called an iteration. The loop in the example above make 5 iterations and second one 3 iterations.

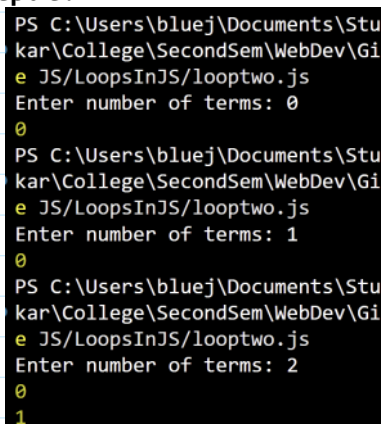
The "do...while" loop

The condition check can be moved below the loop body using the do...while syntax:

```
do{
  //loop body
}while(<condition>);
```

```
const prompt = require('prompt-sync')();
let numOfTerms = Number(prompt(`Enter number of terms: `))
let i = 0;
do{
  console.log(i);
  i++;
}while(i < numOfTerms);
```

Output:



```
PS C:\Users\bluej\Documents\Stu
kar\College\SecondSem\WebDev\Gi
e JS/LoopsInJS/looptwo.js
Enter number of terms: 0
0
PS C:\Users\bluej\Documents\Stu
kar\College\SecondSem\WebDev\Gi
e JS/LoopsInJS/looptwo.js
Enter number of terms: 1
0
PS C:\Users\bluej\Documents\Stu
kar\College\SecondSem\WebDev\Gi
e JS/LoopsInJS/looptwo.js
Enter number of terms: 2
0
1
```

This form of syntax should only be used when you want the body of the loop to execute at least once, regardless of the condition being truthy. Usually the other form is preferred `while(...){...}`

The "for" loop

The for loop is more complex, but it's also the most commonly used loop.

Syntax:

```
for(begin; condition; step){
  //loop body
}
```

For example:

```
const prompt = require('prompt-sync')();
let numOfTerms = Number(prompt(`Enter number of terms: `))
for(let i = 0; i < numOfTerms; i++){
  console.log(i);
}
```

Output:

```
e JS/LoopsInJS/forloopone.js
Enter number of terms: 0
PS C:\Users\bluej\Documents\St
kar\College\SecondSem\WebDev\G
e JS/LoopsInJS/forloopone.js
Enter number of terms: 1
0
PS C:\Users\bluej\Documents\St
kar\College\SecondSem\WebDev\G
e JS/LoopsInJS/forloopone.js
Enter number of terms: 2
0
1
```

Let's examine the for statement part-by-part

Part	Statement	Description
begin	<code>let i = 0</code>	Executes Once Upon entering the loop.
condition	<code>i < numOfTerms</code>	Checked before every loop iteration. If false, the loop Stops.
body	<code>console.log(i)</code>	It runs again and again while the condition is truthy.
step	<code>i++</code>	Executes after the body on each iteration.

That is begin. Executes once and then it iterates after each condition, test body and step are executed.

Suggestion:

If you are new to loops, it could help to go back to the example and reproduce how it runs step by step on a piece of paper.

```
const prompt = require('prompt-sync')();
let numOfTerms = Number(prompt(`Enter number of terms: `));
for(let i = 0; i < numOfTerms; i++){
  console.log(i);
}
console.log(i)
```

Output:

```
e JS/LoopsInJS/forloopone.js
Enter number of terms: 3
0
1
2
C:\Users\bluej\Documents\Students\DeeptaNarayanSarkar
\College\SecondSem\WebDev\Git\Coding\Project0\JS\Loop
sInJS\forloopone.js:9
console.log(i)
      ^
ReferenceError: i is not defined
    at Object.<anonymous> (C:\Users\bluej\Documents\St
udents\DeeptaNarayanSarkar\College\SecondSem\WebDev\
Git\Coding\Project0\JS\LoopsInJS\forloopone.js:9:13)
```

defined inside the loop block. When JS exit the loop then it will wipe/delete local variable. Therefore, we get Reference error.

Solution of above problem:

```
const prompt = require('prompt-sync')();
```

Output:

The for loop ***begin, condition and step*** are optional, if *condition* is not provided by default JS make it truthy. And the loop becomes infinite, i.e., never ending loop. We have to forcefully stop the code.

Output:

```

    console.log(i);
    i++;
}
console.log(`outside the loop i = ${i}`)

```

Output:

```

e JS/LoopsInJS/forloopone.js
Enter number of terms: 2
0
1
outside the loop i = 2

```

Breaking the loop

Normally a loop exists when its condition becomes falsy.

But we can force the exit at any time using a special break directive.

For example. Loop below ask the user for a series of numbers. Breaking when no number is entered.

```

const prompt = require('prompt-sync')();
let sum = 0;
while(true){
    let value = +prompt('Enter a number: ', '');
    if(!value){
        break;
    }
    sum += value;
}
console.log(`Sum = ${sum}`)

```

← The break directive is activated if the user enters an empty line or cancels the input. It stops the loop immediately, passing control to the first line after the loop. i.e. console.log('Sum = \${sum}')

Output:

```

Enter a number: 5
Enter a number: 7
Enter a number: -9
Enter a number: -2
Enter a number: 9
Enter a number:
Sum = 10

```

The combination "infinite loop + break as needed" is great for situation when the loop's condition must be checked not in the beginning or end of the loop, but in the middle or even in several places of its body.

Continue to the next iteration

The **continue** directive is a lighter version of **break**. It doesn't stop the whole loop. Instead it stops the current iteration and forces the loop to start a new one (if the condition Allows)

We can use it if we are done with the current iteration and would like to move to the next one.

The loop below uses continue to output only odd values:

```

const prompt = require('prompt-sync')();
let i = 0;
let output = '';
let n = +prompt('Enter number of terms: ');
for (; i < n; i++) {
    if (i % 2 == 0) {
        continue;
    }
    output += i + ' ';
}
console.log(output);

```

Output:

```

Enter number of terms: 20
1 3 5 7 9 11 13 15 17 19

```

For even values of I the continue directive stops executing the body and passes control to the next iteration of four (with the next number) So the statement `output += i + ' '` is executed for odd value of i.

Alternative code, not using continue statement:

```

const prompt = require('prompt-sync')();
let i = 0;
let output = '';
let n = +prompt('Enter number of terms: ');
for (; i < n; i++) {
    if (i % 2) {
        output += i + ' ';
    }
}
console.log(output);

```

Output:

```

Enter number of terms: 20
1 3 5 7 9 11 13 15 17 19

```

Labels for break/continue

Sometimes we need to break out from multiple nested loops at once.

```

const prompt = require('prompt-sync')();
outer: for(let i = 0; i < 3; i++){
    for(let j = 0; j < 3; j++){
        let input = prompt(`Value at coords (${i},${j})`, '');
        // If an empty string or canceled, then break out of both loops
        if(!input){
            break outer;
        }
    }
}

```

```
}  
console.log(`done`)
```

Output:

```
Value at coords (0,0)12  
Value at coords (0,1)10  
Value at coords (0,2)11  
Value at coords (1,0)  
done
```

Syntax:

```
break <labelName>: for(...){  
  ...  
}
```

In the code above, *break outer* looks upwards for the label name *outer* and breaks out of that loop. So the control goes straight from *break outer* to `console.log(`done`)`.